Министерство науки и высшего образования Российской Федерации
Пермский филиал федерального государственного бюджетного
образовательного учреждения высшего образования
**«Российский экономический университет имени Г.В. Плеханова»**

# СОПРОВОДИТЕЛЬНАЯ ЗАПИСКА

к практической работе № 2

«Разработка программы “Калькулятор” на C# с использованием технологии WPF, библиотеки классов и модульных тестов»

Выполнил:
студент группы ИПо-33
Давыдов Сергей Павлович

Пермь 2026

# ОГЛАВЛЕНИЕ

ВВЕДЕНИЕ.....3
1 ПОСТАНОВКА ЗАДАЧИ.....4
2 СТРУКТУРА РЕШЕНИЯ.....5
3 СОЗДАНИЕ РЕШЕНИЯ В VISUAL STUDIO.....6
3.1 Создание проекта приложения WPF.....6
3.2 Добавление библиотеки классов.....7
3.3 Добавление проекта модульных тестов.....9
3.4 Добавление ссылок между проектами.....10
3.5 Итоговая структура решения.....11
4 РАЗРАБОТКА БИБЛИОТЕКИ КЛАССОВ.....12
4.1 Класс Calculator.....12
4.2 Класс ExpressionCalculator.....12
5 РАЗРАБОТКА ИНТЕРФЕЙСА WPF.....15
5.1 Разметка окна.....15
5.2 Модель представления.....15
6 МОДУЛЬНОЕ ТЕСТИРОВАНИЕ.....17
7 ПРОВЕРКА РАБОТЫ ПРОГРАММЫ.....19
8 РАЗМЕЩЕНИЕ ПРОЕКТА НА GITHUB.....22
ЗАКЛЮЧЕНИЕ.....23
ПРИЛОЖЕНИЕ А. ИСХОДНЫЙ КОД БИБЛИОТЕКИ КЛАССОВ.....24

# ВВЕДЕНИЕ

Целью практической работы является разработка в среде Microsoft Visual Studio программы «Калькулятор» с графическим интерфейсом на основе технологии WPF (Windows Presentation Foundation). Программа должна выполнять арифметические вычисления с использованием операторов *, /, +, - и ^ (возведение в степень).

По заданию основная логика приложения должна находиться в библиотеке классов, а интерфейс – в приложении WPF. Для классов библиотеки нужно написать модульные тесты. Название проекта содержит фамилию студента в английской транскрипции – **Davydov**.

Для выполнения работы использовались:

– среда разработки Microsoft Visual Studio Community;

– язык программирования C# и платформа .NET 8.0;

– технология WPF и язык разметки XAML;

– платформа модульного тестирования MSTest;

– сервис GitHub для размещения исходного кода.

# 1 ПОСТАНОВКА ЗАДАЧИ

Необходимо разработать программу «Калькулятор» со следующими возможностями:

1) сложение (+), вычитание (-), умножение (*), деление (/) и возведение в степень (^);
2) ввод выражения с помощью кнопок на экране и с клавиатуры;
3) вычисление выражения с учётом приоритета операций: сначала возведение в степень, затем умножение и деление, затем сложение и вычитание;
4) скобки для изменения порядка действий, в том числе вложенные;
5) работа с дробными и отрицательными числами;
6) вывод сообщения об ошибке при делении на ноль или неправильно введённом выражении.

Например, для выражения `2+3*4` программа должна сначала выполнить умножение, а затем сложение и получить результат 14, а для выражения `(2+3)*4` – сначала сложение в скобках и получить 20.

# 2 СТРУКТУРА РЕШЕНИЯ

Решение `DavydovCalculator` состоит из трёх проектов (таблица 1). Приложение WPF и проект тестов подключают библиотеку классов через ссылку на проект.

Таблица 1 – Проекты решения

| Проект | Тип проекта | Назначение |
|---|---|---|
| `DavydovCalculator.Core` | Библиотека классов | Логика калькулятора: операции и вычисление выражения |
| `DavydovCalculator.Wpf` | Приложение WPF | Окно калькулятора с кнопками |
| `DavydovCalculator.Tests` | Тестовый проект MSTest | Модульные тесты для классов библиотеки |

В библиотеке два класса (таблица 2).

Таблица 2 – Классы библиотеки

| Класс | Назначение |
|---|---|
| `Calculator` | Выполняет одну операцию над двумя числами: методы `Add`, `Subtract`, `Multiply`, `Divide`, `Power` и общий метод `Calculate` |
| `ExpressionCalculator` | Вычисляет целое выражение, например `(2+3)*4`, с учётом скобок и приоритета операций. Для каждой операции вызывает класс `Calculator` |

Приложение WPF сделано по шаблону MVVM: окно `MainWindow` отвечает только за внешний вид, а данные и команды находятся в классе `MainViewModel`. Окно связано с ним через привязки данных (Binding).

# 3 СОЗДАНИЕ РЕШЕНИЯ В VISUAL STUDIO

## 3.1 Создание проекта приложения WPF

После запуска Visual Studio выбираем пункт **«Создание проекта»** (или меню **Файл → Создать → Проект**). В строке поиска вводим «WPF» и выбираем шаблон **«Приложение WPF»** для языка C# (рисунок 1), нажимаем **«Далее»**.

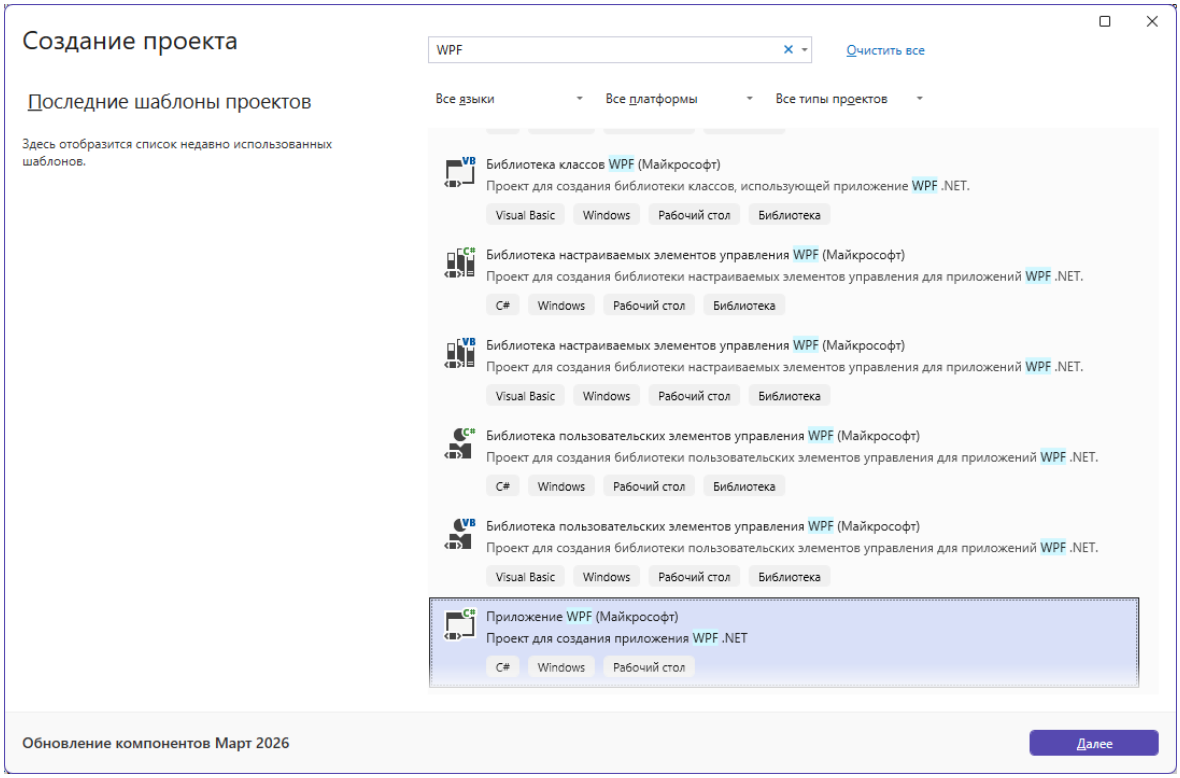



Рисунок 1 – Выбор шаблона «Приложение WPF»

В окне **«Настроить новый проект»** вводим имя проекта `DavydovCalculator.Wpf`, выбираем папку и задаём имя решения `DavydovCalculator` (рисунок 2). Флажок «Поместить решение и проект в одном каталоге» не ставим, так как в решении будет три проекта.

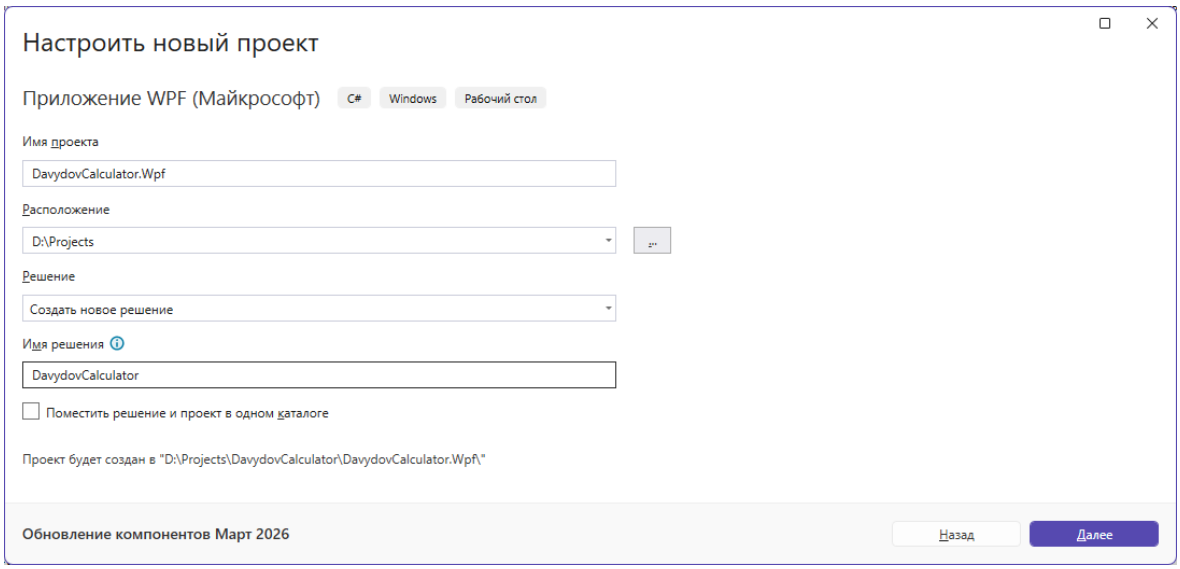



Рисунок 2 – Настройка имени проекта и решения

На странице **«Дополнительные сведения»** выбираем платформу **.NET 8.0** (рисунок 3) и нажимаем **«Создать»**. Visual Studio создаёт решение с проектом, в котором уже есть окно `MainWindow.xaml`.

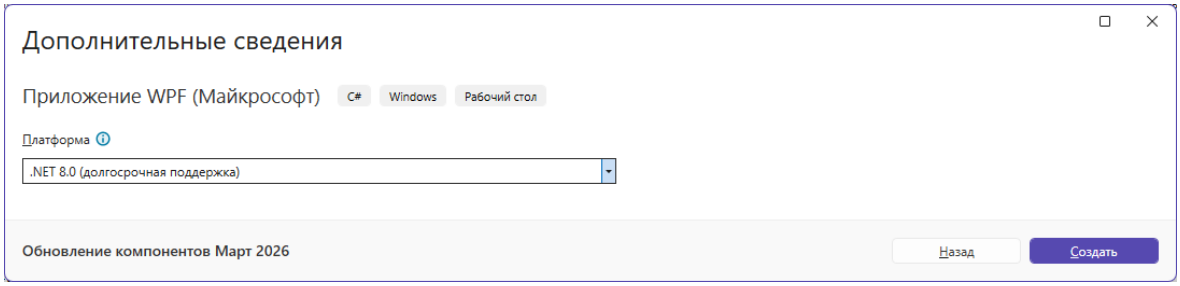



Рисунок 3 – Выбор платформы .NET 8.0

### 3.2 Добавление библиотеки классов

В обозревателе решений щёлкаем правой кнопкой мыши по решению и выбираем **Добавить → Создать проект**. Находим шаблон **«Библиотека классов»** для C# (рисунок 4).

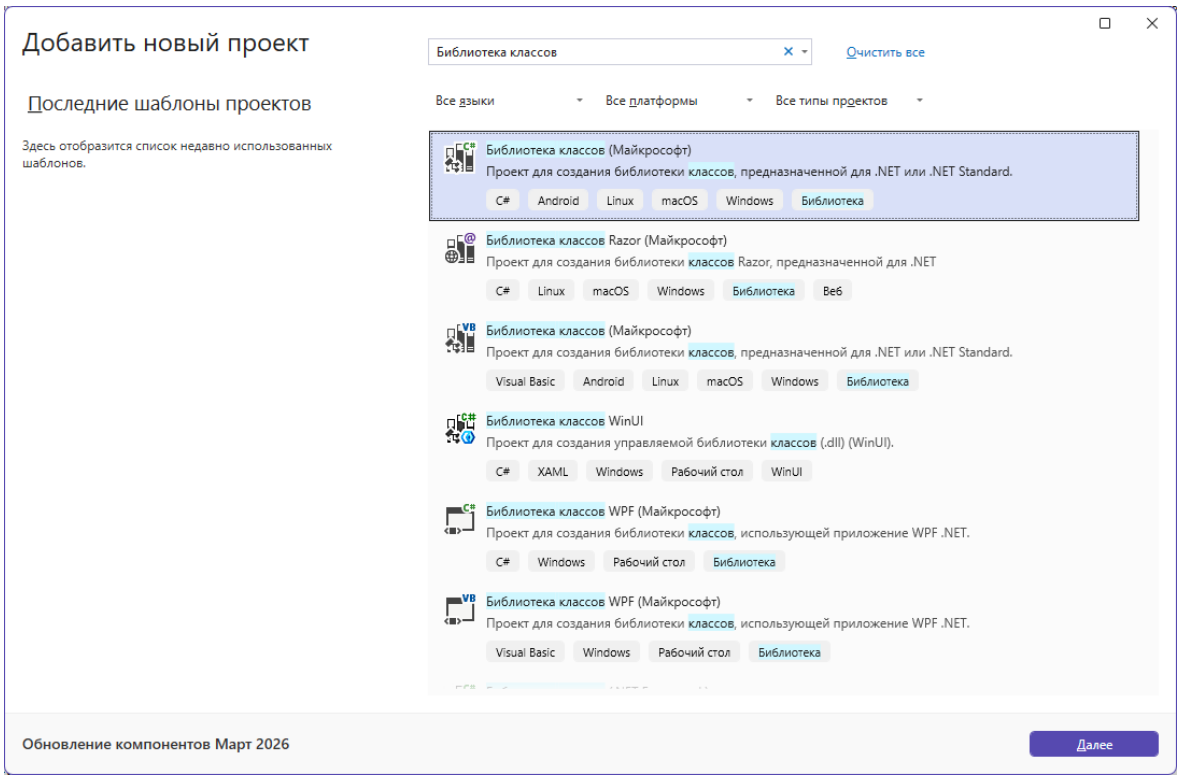



Рисунок 4 – Выбор шаблона «Библиотека классов»

Задаём имя `DavydovCalculator.Core` (рисунок 5), выбираем платформу .NET 8.0 (рисунок 6) и нажимаем **«Создать»**. Файл `Class1.cs`, созданный по шаблону, удаляем.

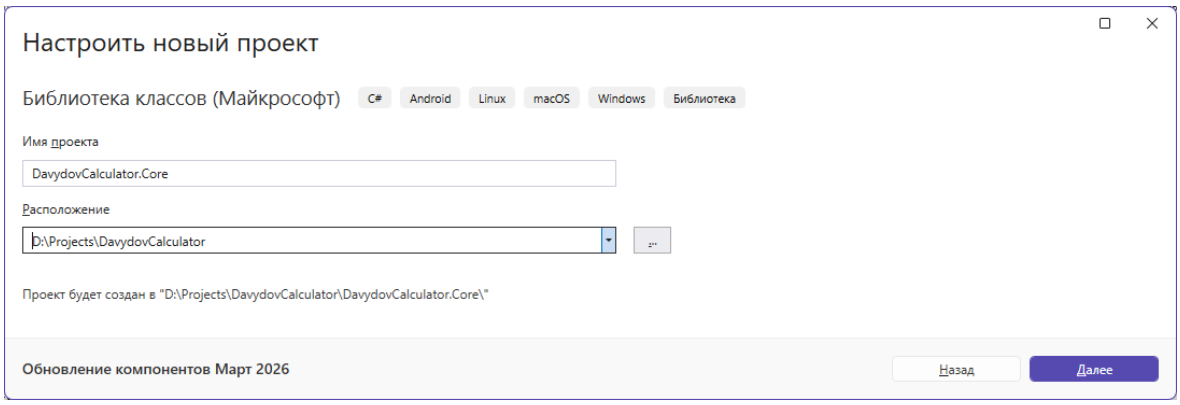



Рисунок 5 – Имя проекта библиотеки классов

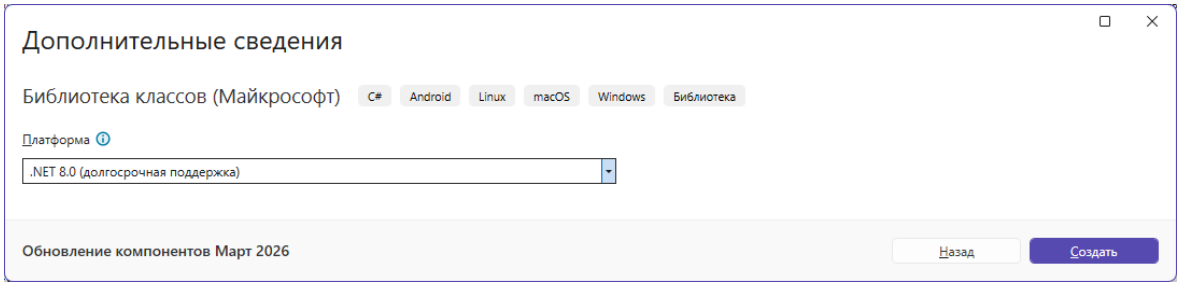



Рисунок 6 – Выбор платформы для библиотеки классов

### 3.3 Добавление проекта модульных тестов

Так же добавляем проект тестов: **Добавить → Создать проект**, шаблон **«Тестовый проект MSTest»** для C# (рисунок 7).

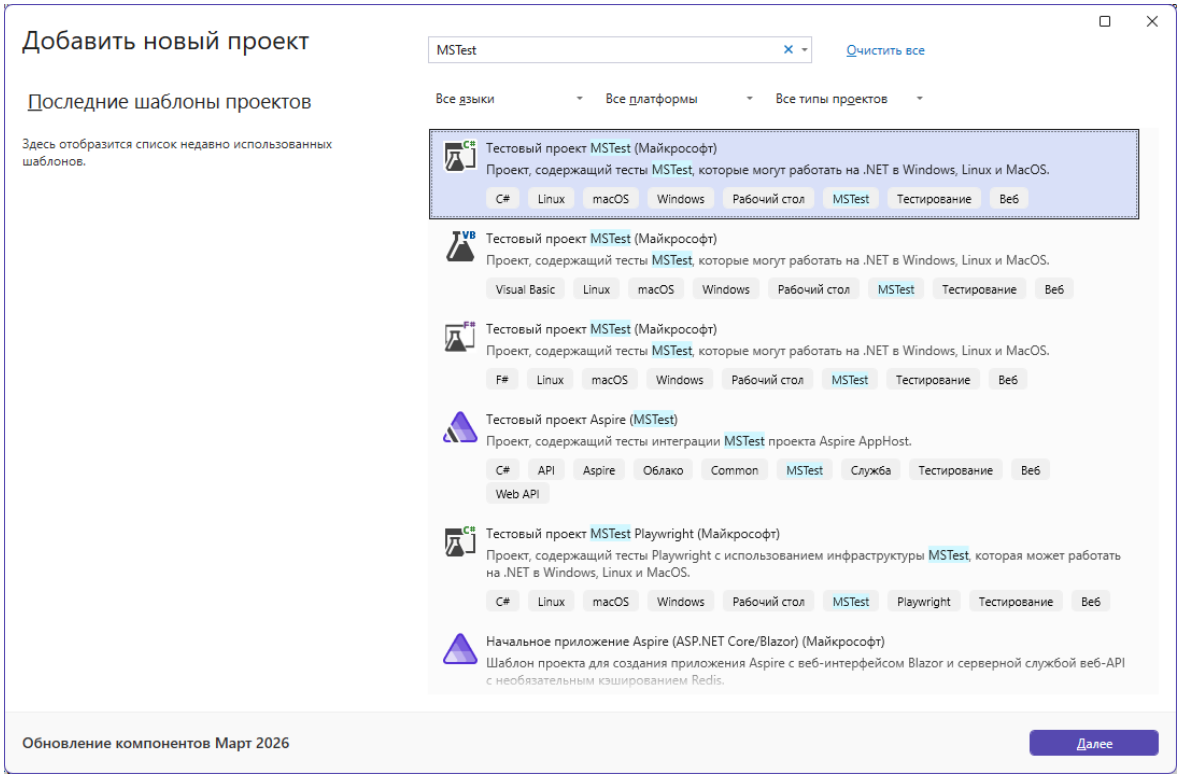



Рисунок 7 – Выбор шаблона «Тестовый проект MSTest»

Задаём имя `DavydovCalculator.Tests` (рисунок 8), выбираем платформу .NET 8.0, остальные параметры оставляем по умолчанию (рисунок 9). Файл `Test1.cs` из шаблона удаляем.

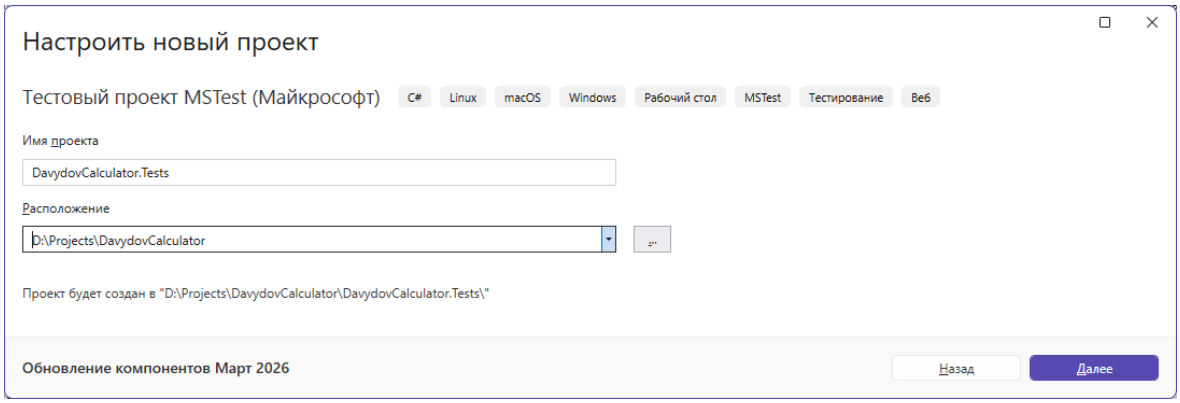



Рисунок 8 – Имя проекта модульных тестов

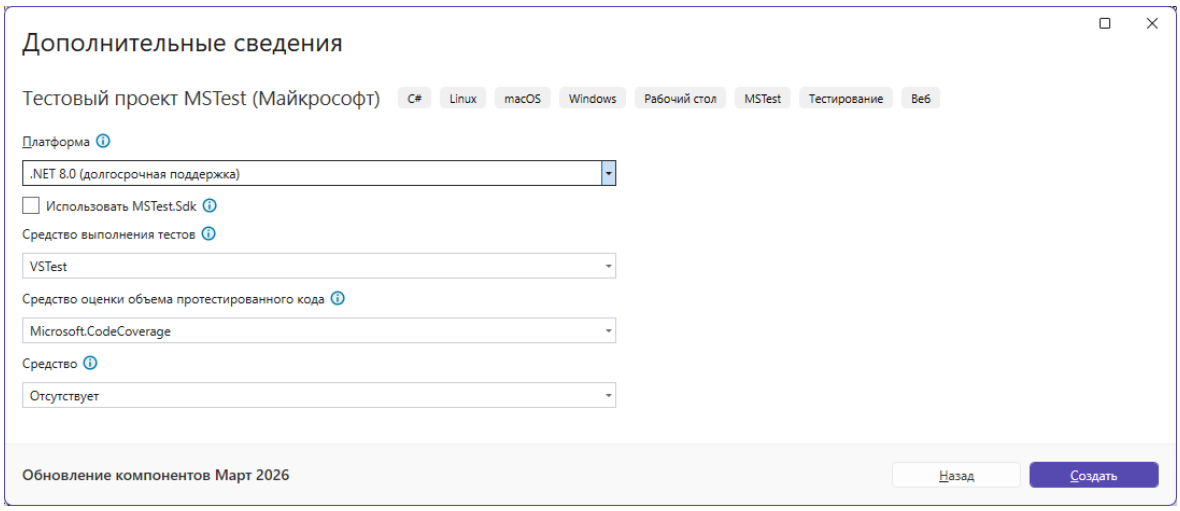



Рисунок 9 – Дополнительные сведения проекта тестов

### 3.4 Добавление ссылок между проектами

Чтобы приложение WPF могло использовать классы библиотеки, щёлкаем правой кнопкой мыши по проекту `DavydovCalculator.Wpf` и выбираем **Добавить → Ссылка на проект**. В окне **«Диспетчер ссылок»** отмечаем `DavydovCalculator.Core` и нажимаем **«ОК»** (рисунок 10).

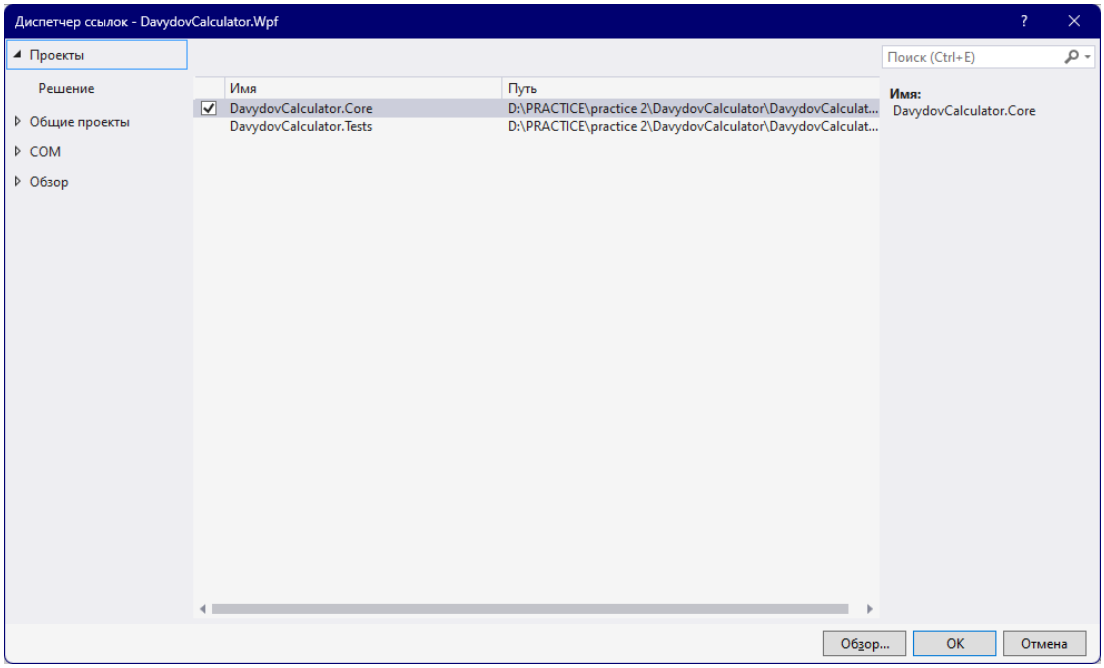



Рисунок 10 – Добавление ссылки на библиотеку классов

Такую же ссылку добавляем в проект `DavydovCalculator.Tests`. Проект `DavydovCalculator.Wpf` назначаем запускаемым (правая кнопка мыши → **«Назначить запускаемым проектом»**), чтобы по клавише **F5** открывалось окно калькулятора.

### 3.5 Итоговая структура решения

Итоговая структура решения показана на рисунке 11. В библиотеке находятся классы `Calculator` и `ExpressionCalculator`, в проекте тестов – по одному тестовому классу на каждый из них, в приложении WPF – окно, модель представления и класс команды.

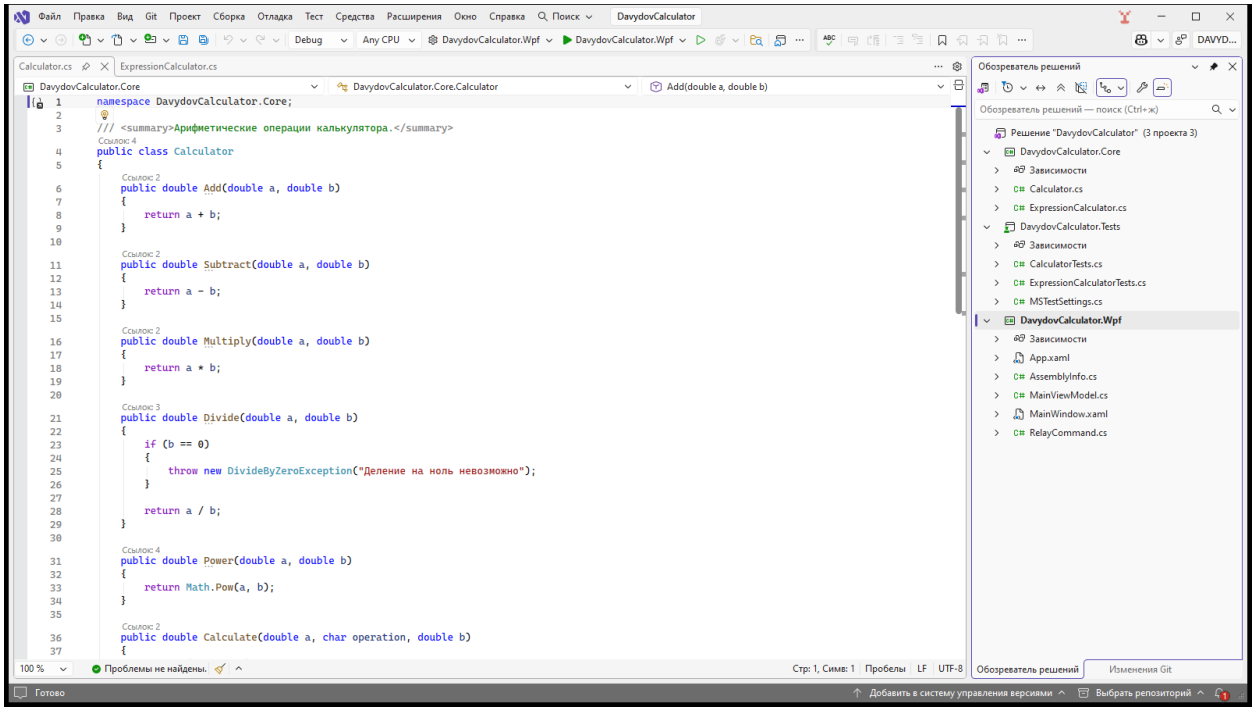



Рисунок 11 – Структура решения и класс Calculator

# 4 РАЗРАБОТКА БИБЛИОТЕКИ КЛАССОВ

## 4.1 Класс Calculator

Класс `Calculator` (рисунок 11) содержит по одному методу на каждую операцию: `Add` – сложение, `Subtract` – вычитание, `Multiply` – умножение, `Divide` – деление, `Power` – возведение в степень (через стандартный метод `Math.Pow`). В методе `Divide` перед делением проверяется делитель, и если он равен нулю, выбрасывается исключение:

```
public double Divide(double a, double b)
{
    if (b == 0)
    {
        throw new DivideByZeroException("Деление на ноль невозможно");
    }

    return a / b;
}
```

Метод `Calculate(a, operation, b)` с помощью оператора `switch` выбирает нужный метод по символу операции. Если символ неизвестен, выбрасывается `ArgumentException`.

## 4.2 Класс ExpressionCalculator

Класс `ExpressionCalculator` вычисляет всё выражение, введённое пользователем (рисунок 12). Вычисление проходит в несколько шагов.

**Шаг 1.** Метод `SplitExpression` проходит по строке посимвольно и делит её на три списка: числа, знаки чисел (1 или −1) и операции. Цифры и запятая добавляются к текущему числу, а когда встречается знак операции, число заканчивается. Минус в начале выражения или сразу после другого знака операции – это минус перед числом (`-5+3`, `2*-3`). Он не добавляется к самому числу, а запоминается в списке знаков. Если встречается недопустимый символ или число не удаётся прочитать (например, выражение заканчивается знаком операции), выбрасывается `FormatException`.

**Скобки.** Когда метод `SplitExpression` встречает открывающую скобку, метод `FindClosingBracket` находит парную ей закрывающую (с

учётом вложенных скобок). Выражение внутри скобок вычисляется отдельно тем же методом `Calculate`, а его результат дальше используется как обычное число (рисунок 13). Поэтому вложенные скобки, например `2*(1+(5-2))`, вычисляются автоматически: сначала `5-2 = 3`, затем `1+3 = 4`, затем `2*4 = 8`. Так как результат скобок – это готовое число, `(-2)^2 = 4`, а без скобок `-2^2 = -4`. Если скобка не закрыта, закрыта лишний раз, пустая или перед ней пропущен знак операции, выбрасывается `FormatException` с понятным сообщением.

**Шаг 2.** Выполняются операции в порядке приоритета. Сначала метод `CalculatePowers` выполняет все возведения в степень (справа налево, как принято в математике: `2^3^2 = 2^9 = 512`). Затем метод `ApplySigns` применяет минусы перед числами. Благодаря такому порядку `-2^2` вычисляется как `-(2^2) = -4`, как в математике. Минус перед показателем степени учитывается сразу, поэтому `2^-2 = 0,25`. После этого метод `CalculateOperations` выполняет умножение и деление, а затем – сложение и вычитание. Каждая выполненная операция заменяет два числа одним результатом и удаляет знак операции из списка.

Например, для выражения `2+3*4`:

– после разбиения: числа [2, 3, 4], операции [+, *];

– после умножения: числа [2, 12], операции [+];

– после сложения: числа [14], операций нет – результат 14.

Для выражения `-2^2+5`:

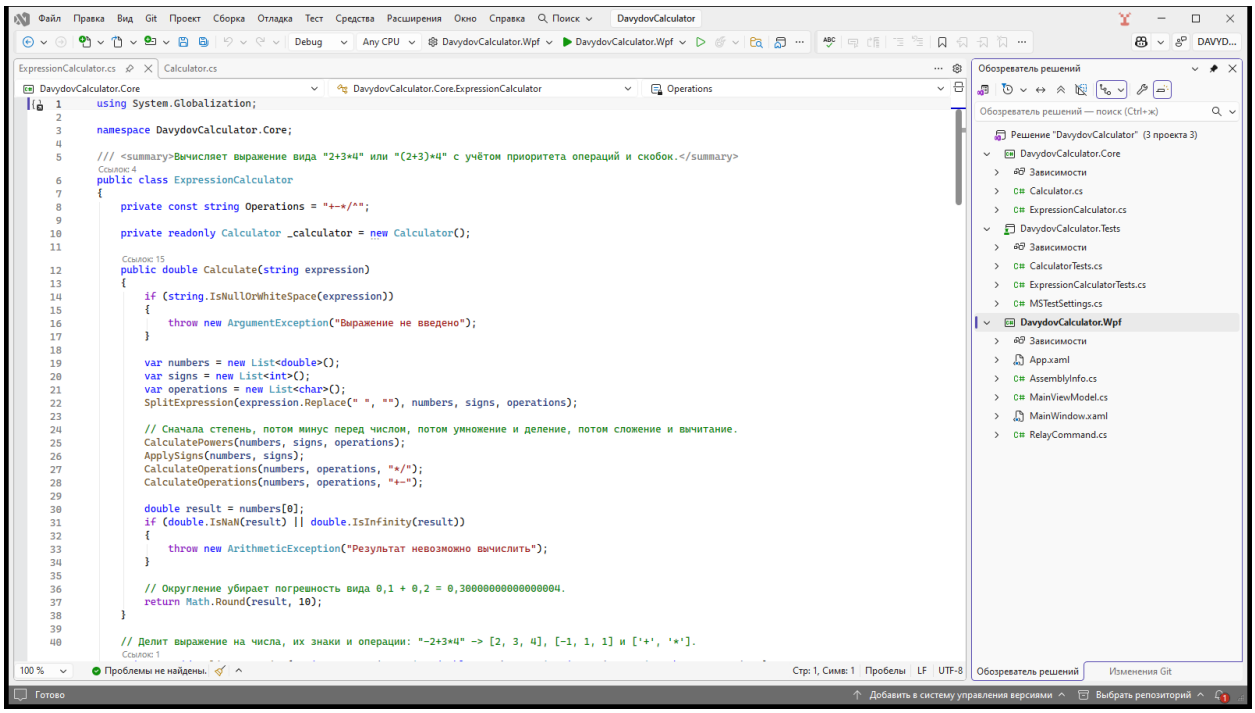
– после разбиения: числа [2, 2, 5], знаки чисел [−1, 1, 1], операции [^, +];

– после возведения в степень: числа [4, 5], знаки [−1, 1], операции [+];

– после применения знаков: числа [−4, 5];

– после сложения: результат 1.

**Шаг 3.** Результат проверяется: если получилось слишком большое число (например, `10^400`), выбрасывается исключение. Затем результат округляется до 10 знаков после запятой, чтобы не показывать погрешность вычислений с дробными числами (`0,1+0,2` даёт `0,3`).



```
using System.Globalization;

namespace DavydovCalculator.Core;

/// <summary>Вычисляет выражение вида "2+3*4" или "(2+3)*4" с учётом приоритета операций и скобок.</summary>
Ссылок: 4
public class ExpressionCalculator
{
    private const string Operations = "+-*/^";

    private readonly Calculator _calculator = new Calculator();

    Ссылок: 15
    public double Calculate(string expression)
    {
        if (string.IsNullOrWhiteSpace(expression))
        {
            throw new ArgumentException("Выражение не введено");
        }

        var numbers = new List<double>();
        var signs = new List<int>();
        var operations = new List<char>();
        SplitExpression(expression.Replace(" ", ""), numbers, signs, operations);

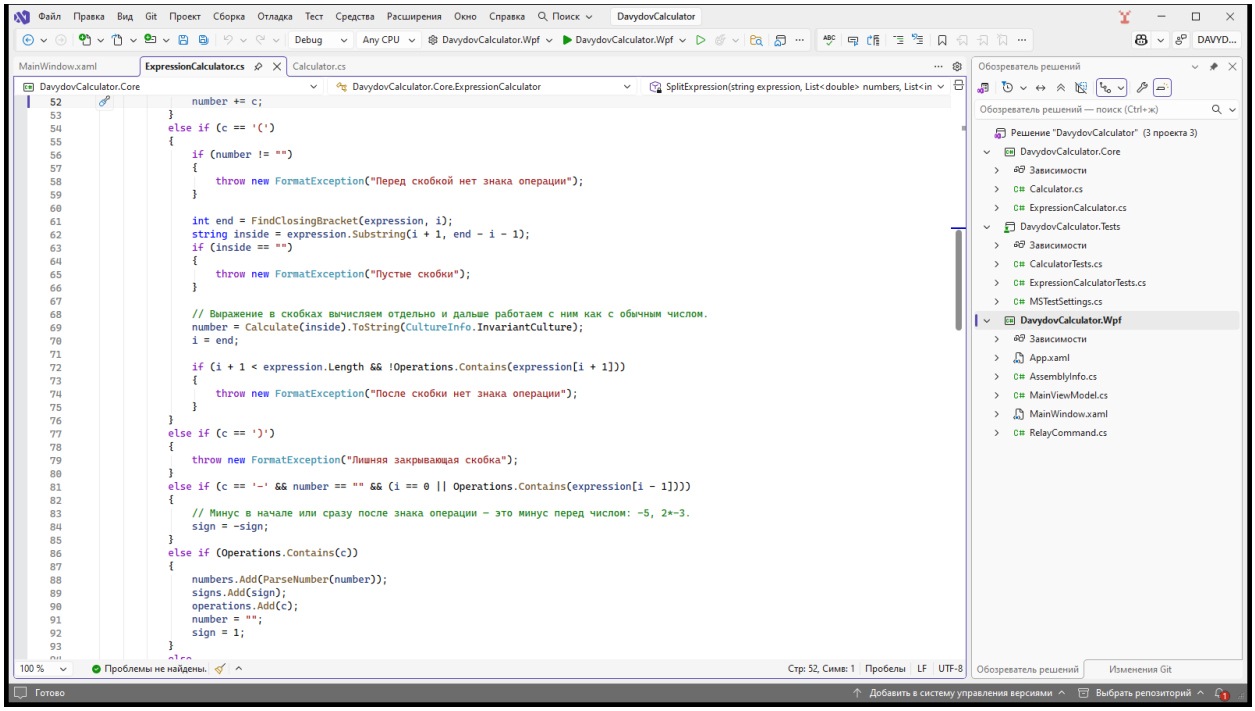
        // Сначала степень, потом минус перед числом, потом умножение и деление, потом сложение и вычитание.
        CalculatePowers(numbers, signs, operations);
        ApplySigns(numbers, signs);
        CalculateOperations(numbers, operations, "*/");
        CalculateOperations(numbers, operations, "+-");

        double result = numbers[0];
        if (double.IsNaN(result) || double.IsInfinity(result))
        {
            throw new ArithmeticException("Результат невозможно вычислить");
        }

        // Округление убирает погрешность вида 0,1 + 0,2 = 0,30000000000000004.
        return Math.Round(result, 10);
    }

    // Делит выражение на числа, их знаки и операции: "-2+3*4" -> [2, 3, 4], [-1, 1, 1] и ['+', '*'].
    Ссылок: 1
```

Рисунок 12 – Класс ExpressionCalculator



```
            number += c;
        }
        else if (c == '(')
        {
            if (number != "")
            {
                throw new FormatException("Перед скобкой нет знака операции");
            }

            int end = FindClosingBracket(expression, i);
            string inside = expression.Substring(i + 1, end - i - 1);
            if (inside == "")
            {
                throw new FormatException("Пустые скобки");
            }

            // Выражение в скобках вычисляем отдельно и дальше работаем с ним как с обычным числом.
            number = Calculate(inside).ToString(CultureInfo.InvariantCulture);
            i = end;

            if (i + 1 < expression.Length && !Operations.Contains(expression[i + 1]))
            {
                throw new FormatException("После скобки нет знака операции");
            }
        }
        else if (c == ')')
        {
            throw new FormatException("Лишняя закрывающая скобка");
        }
        else if (c == '-' && number == "" && (i == 0 || Operations.Contains(expression[i - 1])))
        {
            // Минус в начале или сразу после знака операции — это минус перед числом: -5, 2*-3.
            sign = -sign;
        }
        else if (Operations.Contains(c))
        {
            numbers.Add(ParseNumber(number));
            signs.Add(sign);
            operations.Add(c);
            number = "";
            sign = 1;
        }
        else
```

Рисунок 13 – Обработка скобок в методе SplitExpression

# 5 РАЗРАБОТКА ИНТЕРФЕЙСА WPF

## 5.1 Разметка окна

Окно описано на языке XAML в файле `MainWindow.xaml` (рисунок 14). Вверху окна два текстовых поля: в первом показывается вводимое выражение, во втором – результат или сообщение об ошибке. Ниже расположена сетка `Grid` из кнопок: цифры, запятая, знаки операций, скобки, «C» (очистить), «⌫» (удалить последний символ) и «=» (вычислить).

Оформление кнопок вынесено в стили в файле `App.xaml`: общий стиль для всех кнопок со скруглёнными углами и отдельные стили для кнопок операций и кнопки «=».

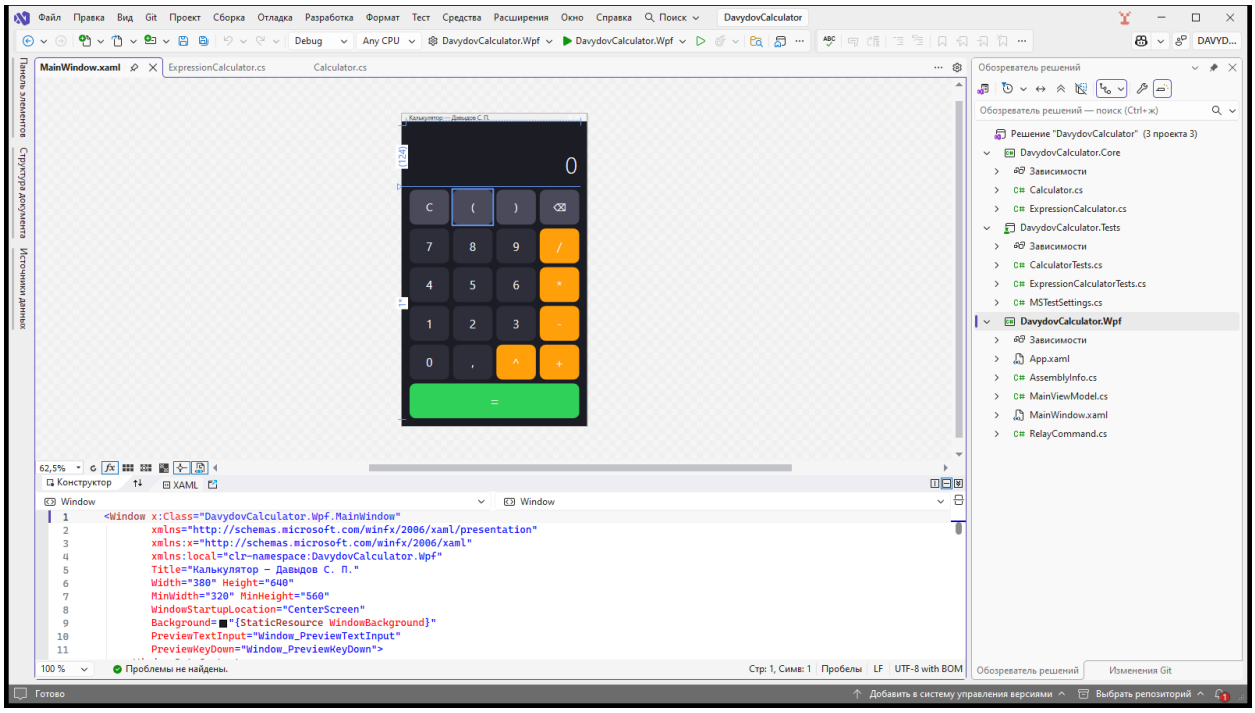



Рисунок 14 – Окно калькулятора в конструкторе XAML

## 5.2 Модель представления

Класс `MainViewModel` содержит свойства `Expression` (выражение) и `Result` (результат) и команды для кнопок. Класс реализует интерфейс `INotifyPropertyChanged`, поэтому при изменении свойства текст в окне обновляется автоматически. Команды созданы с помощью класса `RelayCommand`, который реализует интерфейс `ICommand`.

Все кнопки цифр и операций привязаны к одной команде AddSymbolCommand, а символ передаётся через параметр:

```
<Button Content="7" Style="{StaticResource CalcButton}"
        Command="{Binding AddSymbolCommand}" CommandParameter="7" />
```

Кнопка «=» вызывает метод Calculate, который передаёт выражение в класс ExpressionCalculator из библиотеки. Если при вычислении возникает исключение, вместо результата выводится текст ошибки:

```
public void Calculate()
{
    try
    {
        Result = _calculator.Calculate(Expression).ToString();
    }
    catch (Exception ex)
    {
        Result = "Ошибка: " + ex.Message;
    }
}
```

Для ввода с клавиатуры в файле MainWindow.xaml.cs обрабатываются события PreviewTextInput (цифры и знаки) и PreviewKeyDown (Enter – вычислить, Backspace – удалить символ, Esc – очистить).

# 6 МОДУЛЬНОЕ ТЕСТИРОВАНИЕ

Для каждого класса библиотеки создан тестовый класс (таблица 3). Тестовые методы отмечены атрибутом `[TestMethod]`. Если один метод нужно проверить на нескольких примерах, используется атрибут `[DataRow]`.

Таблица 3 – Модульные тесты

| Тестовый класс | Тестов | Что проверяется |
|---|---|---|
| `CalculatorTests` | 8 | Результаты всех пяти операций, отрицательная степень, деление на ноль, неизвестная операция |
| `ExpressionCalculatorTests` | 22 | Простые выражения, приоритет операций, скобки (в том числе вложенные и `(-2)^2 = 4`), степень справа налево, минус перед степенью (`-2^2 = -4`), отрицательный показатель, отрицательные и дробные числа, ошибки: деление на ноль, пустое и неправильное выражение, незакрытая и лишняя скобка, слишком большой результат |

Пример тестов:

```
[TestMethod]
public void Divide_ByZero_ThrowsException()
{
    Assert.ThrowsExactly<DivideByZeroException>(() => _calculator.Divide(5, 0));
}


[TestMethod]
[DataRow("2+3*4", 14.0)]
[DataRow("2*3^2", 18.0)]
public void Calculate_OperationPriority_IsRespected(string expression, double
expected)
{
    Assert.AreEqual(expected, _calculator.Calculate(expression));
}
```

Тесты запускаются через меню **Тест → Запуск всех тестов**. Результаты показаны в окне **«Обозреватель тестов»** (рисунок 15): все 30 тестов пройдены успешно.

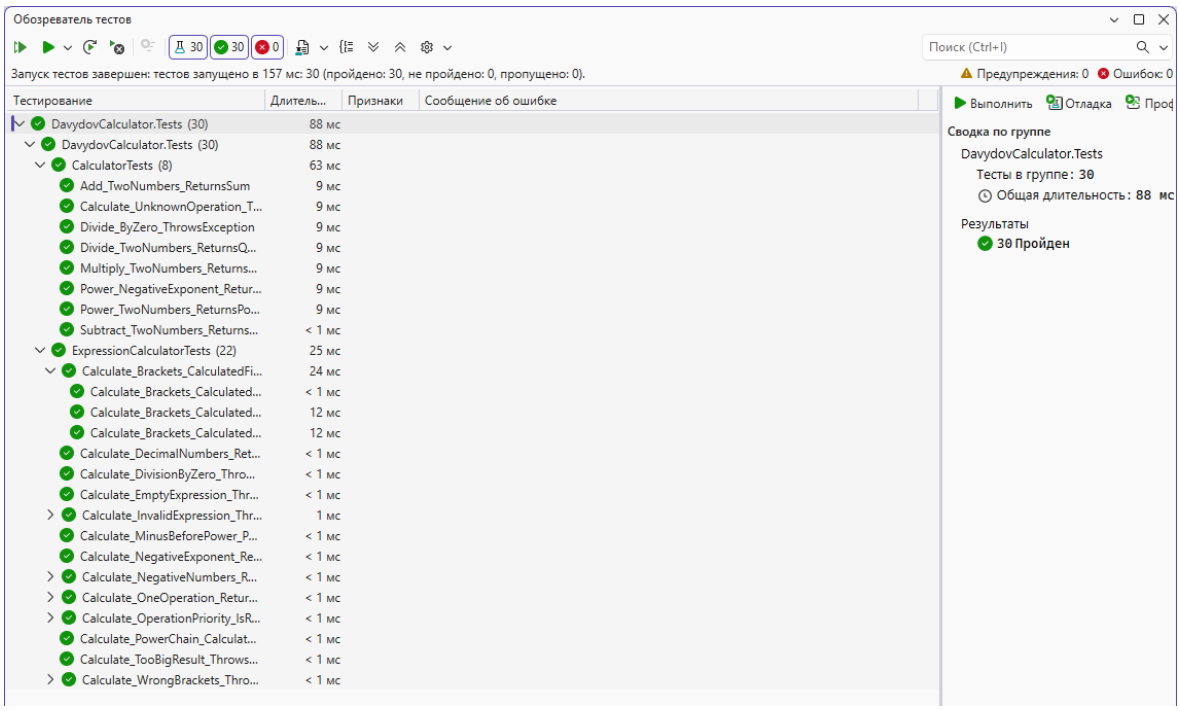



Рисунок 15 – Результаты модульных тестов

# 7 ПРОВЕРКА РАБОТЫ ПРОГРАММЫ

Запускаем программу клавишей **F5**. На рисунке 16 показано окно после запуска, на рисунке 17 – ввод выражения со скобками `(2+3)*4`.

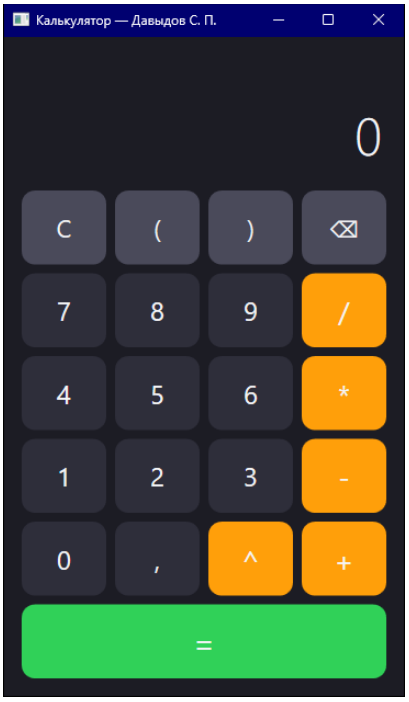



Рисунок 16 – Окно после запуска

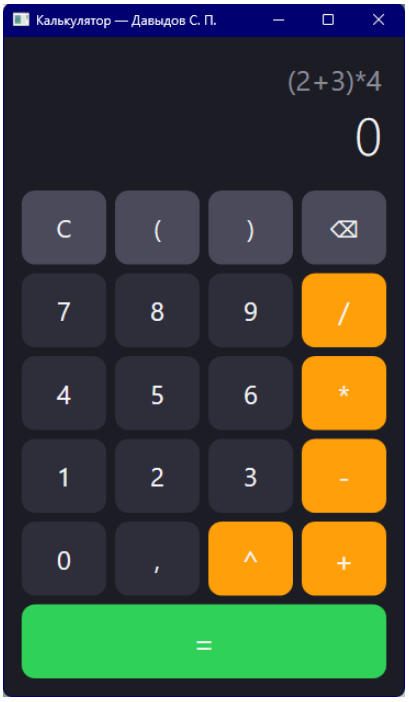



Рисунок 17 – Ввод выражения

После нажатия «=» программа выводит результат 20: сначала выполнено сложение в скобках, затем умножение (рисунок 18). При делении на ноль выводится сообщение об ошибке, программа продолжает работать (рисунок 19).

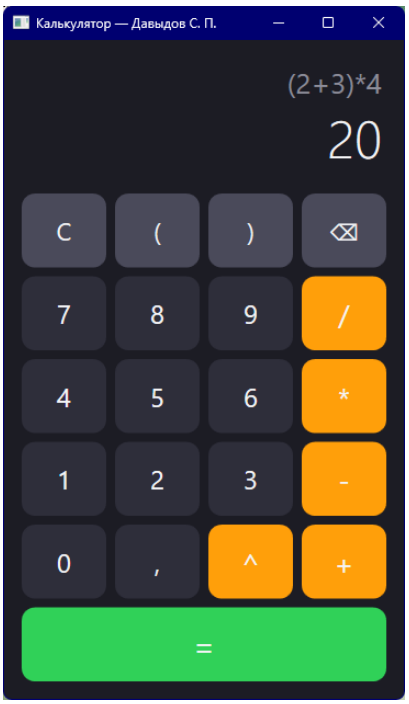



Рисунок 18 – Результат вычисления со скобками

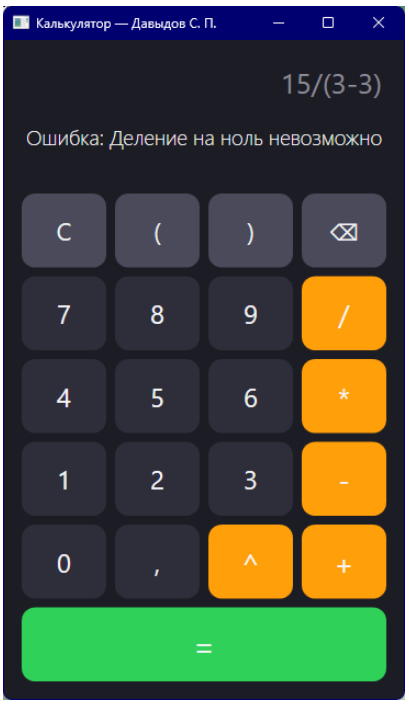



Рисунок 19 – Деление на ноль

На рисунке 20 показано сложение дробных чисел, на рисунке 21 – возведение в степень справа налево: `2^3^2 = 2^9 = 512`.

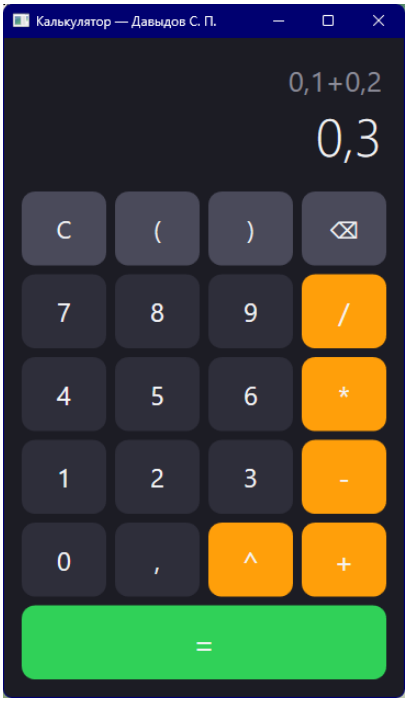



Рисунок 20 – Дробные числа

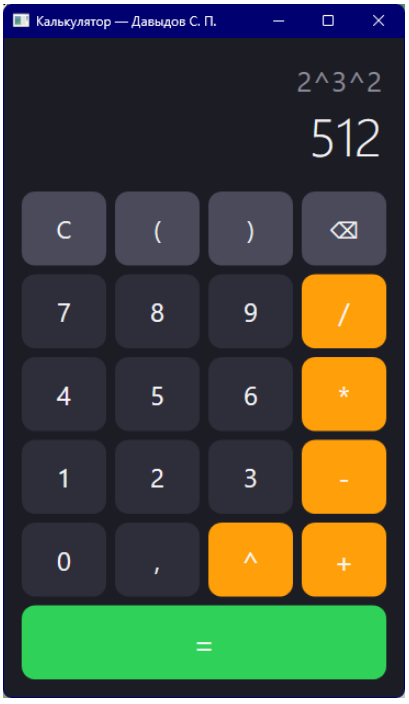



Рисунок 21 – Возведение в степень

На рисунках 22 и 23 показана разница между минусом перед степенью и отрицательным числом в скобках: `-2^2 = -(2^2) = -4`, а `(-2)^2 = 4`, как

в математике. Все результаты совпадают с ожидаемыми, программа работает правильно.

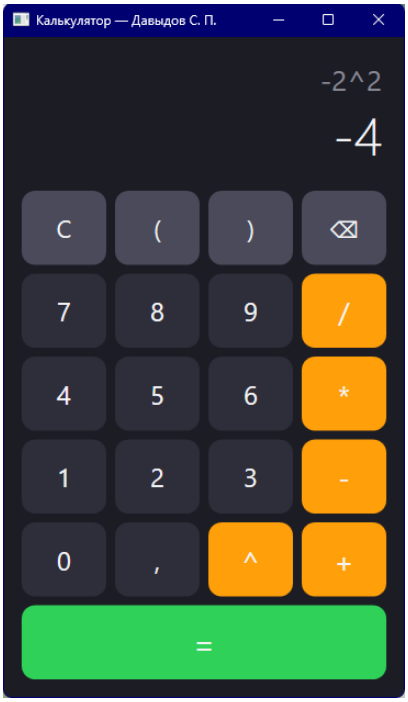



Рисунок 22 – Минус перед степенью

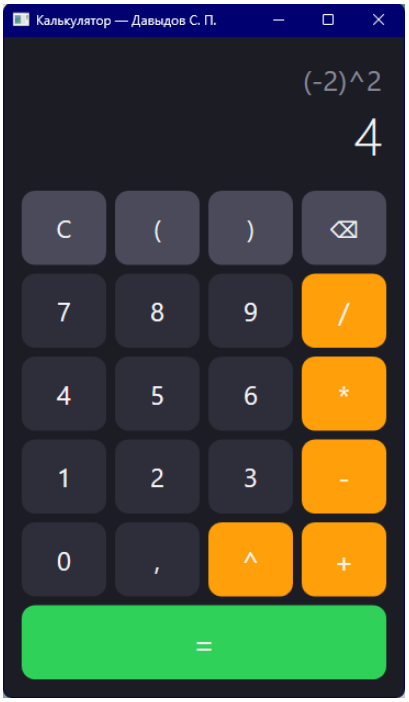



Рисунок 23 – Отрицательное число в скобках

# 8 РАЗМЕЩЕНИЕ ПРОЕКТА НА GITHUB

Для размещения исходного кода на сайте github.com создан публичный репозиторий `DavydovCalculator`. В него загружены:

– файл решения `DavydovCalculator.sln`;

– папки проектов `DavydovCalculator.Core`, `DavydovCalculator.Wpf`, `DavydovCalculator.Tests`;

– файлы `README.md` (описание проекта) и `.gitignore`;

– сопроводительная записка в формате PDF.

Папки `bin`, `obj` и `.vs`, которые Visual Studio создаёт при сборке, не загружаются. Ссылка на репозиторий отправлена преподавателю по электронной почте.

# ЗАКЛЮЧЕНИЕ

В ходе практической работы в Microsoft Visual Studio разработана программа «Калькулятор» на языке C# с интерфейсом WPF. Программа выполняет операции +, -, *, / и ^ с учётом их приоритета и скобок, работает с дробными и отрицательными числами и сообщает об ошибках.

Логика калькулятора вынесена в библиотеку классов `DavydovCalculator.Core`, интерфейс сделан в приложении `DavydovCalculator.Wpf`. Для классов библиотеки написаны модульные тесты в проекте `DavydovCalculator.Tests`, все тесты проходят успешно. Исходный код и записка размещены на GitHub.

# ПРИЛОЖЕНИЕ А. ИСХОДНЫЙ КОД БИБЛИОТЕКИ КЛАССОВ

### Файл Calculator.cs

```
namespace DavydovCalculator.Core;

/// <summary>Арифметические операции калькулятора.</summary>
public class Calculator
{
    public double Add(double a, double b)
    {
        return a + b;
    }

    public double Subtract(double a, double b)
    {
        return a - b;
    }

    public double Multiply(double a, double b)
    {
        return a * b;
    }

    public double Divide(double a, double b)
    {
        if (b == 0)
        {
            throw new DivideByZeroException("Деление на ноль невозможно");
        }

        return a / b;
    }

    public double Power(double a, double b)
    {
        return Math.Pow(a, b);
    }

    public double Calculate(double a, char operation, double b)
    {
        switch (operation)
        {
            case '+':
                return Add(a, b);
            case '-':
                return Subtract(a, b);
            case '*':
                return Multiply(a, b);
            case '/':
                return Divide(a, b);
            case '^':
                return Power(a, b);
            default:
                throw new ArgumentException("Неизвестная операция: " + operation);
        }
    }
}
```

**Файл ExpressionCalculator.cs**

```
using System.Globalization;

namespace DavydovCalculator.Core;

/// <summary>Вычисляет выражение вида "2+3*4" или "(2+3)*4" с учётом приоритета
операций и скобок.</summary>
public class ExpressionCalculator
{
    private const string Operations = "+-*/^";

    private readonly Calculator _calculator = new Calculator();

    public double Calculate(string expression)
    {
        if (string.IsNullOrWhiteSpace(expression))
        {
            throw new ArgumentException("Выражение не введено");
        }

        var numbers = new List<double>();
        var signs = new List<int>();
        var operations = new List<char>();
        SplitExpression(expression.Replace(" ", ""), numbers, signs, operations);

        // Сначала степень, потом минус перед числом, потом умножение и деление, потом
сложение и вычитание.
        CalculatePowers(numbers, signs, operations);
        ApplySigns(numbers, signs);
        CalculateOperations(numbers, operations, "*/");
        CalculateOperations(numbers, operations, "+-");

        double result = numbers[0];
        if (double.IsNaN(result) || double.IsInfinity(result))
        {
            throw new ArithmeticException("Результат невозможно вычислить");
        }

        // Округление убирает погрешность вида 0,1 + 0,2 = 0,30000000000000004.
        return Math.Round(result, 10);
    }

    // Делит выражение на числа, их знаки и операции: "-2+3*4" -> [2, 3, 4], [-1, 1,
1] и ['+', '*'].
    private void SplitExpression(string expression, List<double> numbers, List<int>
signs, List<char> operations)
    {
        string number = "";
        int sign = 1;

        for (int i = 0; i < expression.Length; i++)
        {
            char c = expression[i];

            if (char.IsDigit(c) || c == ',' || c == '.')
            {
                number += c;
            }
            else if (c == '(')
```

```
            {
                if (number != "")
                {
                    throw new FormatException("Перед скобкой нет знака операции");
                }

                int end = FindClosingBracket(expression, i);
                string inside = expression.Substring(i + 1, end - i - 1);
                if (inside == "")
                {
                    throw new FormatException("Пустые скобки");
                }

                // Выражение в скобках вычисляем отдельно и дальше работаем с ним как
с обычным числом.
                number = Calculate(inside).ToString(CultureInfo.InvariantCulture);
                i = end;

                if (i + 1 < expression.Length && !Operations.Contains(expression[i +
1]))
                {
                    throw new FormatException("После скобки нет знака операции");
                }
            }
            else if (c == ')')
            {
                throw new FormatException("Лишняя закрывающая скобка");
            }
            else if (c == '-' && number == "" && (i == 0 ||
Operations.Contains(expression[i - 1])))
            {
                // Минус в начале или сразу после знака операции — это минус перед
числом: -5, 2*-3.
                sign = -sign;
            }
            else if (Operations.Contains(c))
            {
                numbers.Add(ParseNumber(number));
                signs.Add(sign);
                operations.Add(c);
                number = "";
                sign = 1;
            }
            else
            {
                throw new FormatException("Недопустимый символ: " + c);
            }
        }

        numbers.Add(ParseNumber(number));
        signs.Add(sign);
    }

    private static int FindClosingBracket(string expression, int openIndex)
    {
        int depth = 0;

        for (int i = openIndex; i < expression.Length; i++)
        {
            if (expression[i] == '(')
            {
                depth++;
```

```
            }
            else if (expression[i] == ')')
            {
                depth--;
                if (depth == 0)
                {
                    return i;
                }
            }
        }

        throw new FormatException("Не хватает закрывающей скобки");
    }

    private static double ParseNumber(string text)
    {
        text = text.Replace(',', '.');

        if (!double.TryParse(text, NumberStyles.Float, CultureInfo.InvariantCulture,
out double value))
        {
            throw new FormatException("Некорректное выражение");
        }

        return value;
    }

    // Степень считается справа налево: 2^3^2 = 2^9.
    // Минус перед основанием применяется после степени (-2^2 = -4),
    // а минус перед показателем — сразу (2^-2 = 0,25).
    private void CalculatePowers(List<double> numbers, List<int> signs, List<char>
operations)
    {
        for (int i = operations.Count - 1; i >= 0; i--)
        {
            if (operations[i] == '^')
            {
                double exponent = signs[i + 1] * numbers[i + 1];
                numbers[i] = _calculator.Power(numbers[i], exponent);
                numbers.RemoveAt(i + 1);
                signs.RemoveAt(i + 1);
                operations.RemoveAt(i);
            }
        }
    }

    private static void ApplySigns(List<double> numbers, List<int> signs)
    {
        for (int i = 0; i < numbers.Count; i++)
        {
            numbers[i] = signs[i] * numbers[i];
        }
    }

    private void CalculateOperations(List<double> numbers, List<char> operations,
string symbols)
    {
        int i = 0;
        while (i < operations.Count)
        {
            if (symbols.Contains(operations[i]))
            {
```

```
                numbers[i] = _calculator.Calculate(numbers[i], operations[i],
numbers[i + 1]);
                numbers.RemoveAt(i + 1);
                operations.RemoveAt(i);
            }
            else
            {
                i++;
            }
        }
    }
}
```